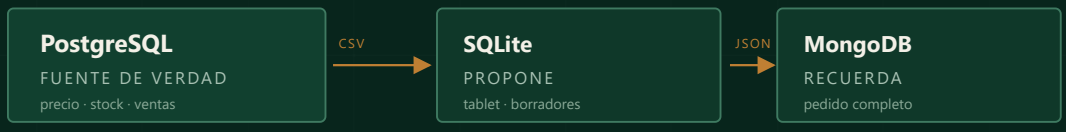
TRABAJO PRÁCTICO INTEGRADOR · SEGUNDA ENTREGA

Food Store

La capa de datos completa: relacional, documental y móvil

Informe técnico consolidado de las unidades **4 a 8**: normalización avanzada y redundancia, administración y seguridad, mantenimiento y recuperación, NoSQL e integración con dispositivos móviles.



EQUIPO DE TRABAJO

Sabrina Gimenez · Ismael Saleme
Jeronimo Coronel · Joaquin Godoy

MATERIA Y MOTORES

Base de Datos II · UTN TUP
PostgreSQL 17
MongoDB · SQLite 3.50

ENTREGA

Noviembre de 2026
Unidades 4 a 8 · TPI «Food Store»

github.com/ismaSaleme12/02_Base-de-datos-II · github.com/ismaSaleme12/Proyecto_Food_Store_MongoDB_SqlLite

SÍNTESIS DE LA SEGUNDA ENTREGA

La capa de datos quedó cerrada.

Este informe consolida el trabajo de las unidades 4 a 8 del TPI «Food Store» a partir de los dos repositorios del equipo. No es un resumen de intenciones: cada afirmación se apoya en un script, una salida real o una medición que puede reproducirse. La tesis que ordena todo el documento es una sola: **PostgreSQL decide, MongoDB recuerda cómo se veía y SQLite propone.**

3	5	319×	48,3 s
MOTORES INTEGRADOS POSTGRESQL · MONGODB · SQLITE	UNIDADES CUBIERTAS U4 A U8	MEJORA DEL REPORTE DESNORMALIZADO (U4)	RTO OBSERVADO SIMULACRO DE DESASTRE (U6)

Qué se hizo, en una página

La **Unidad 4** refactorizó un esquema heredado con redundancia hasta BCNF, migró la instancia sin pérdida y documentó una desnormalización controlada que reduce el reporte de ventas de 73,9 ms a 0,23 ms, con auditoría de sincronía en cero filas.

La **Unidad 5** diseñó un esquema de seis roles con mínimo privilegio, lo verificó con `SET ROLE` real, activó auditoría de conexiones y sentencias y reconstruyó un incidente de acceso indebido a partir del log, con contención decidida por el equipo.

La **Unidad 6** automatizó respaldo, salud y mantenimiento, y ejecutó un simulacro de desastre contra reloj: restauración completa en **48,3 segundos**, con conteos y restricciones lógicas verificadas.

La **Unidad 7** modeló el pedido completo como documento en MongoDB, migró 10 pedidos reales y comparó las mismas consultas contra PostgreSQL: coinciden cantidad e identificadores. La conclusión es mantener PostgreSQL como fuente de verdad y MongoDB como proyección de lectura.

La **Unidad 8** construyó la base SQLite de la tablet, el semáforo de sincronización y seis casos de conflicto. La prueba final demuestra que el móvil leyó y no escribió: pedidos y stock idénticos a la línea de base, con huella md5; el precio sí cambió, y eso es exactamente lo que se quería probar.

EL HALLAZGO QUE ATRAVIESA EL INFORME

La IA acelera la producción, pero **no es el árbitro de la verdad**. Cinco defectos relevantes del trabajo no se detectaron leyendo: se detectaron ejecutando. El más peligroso fue silencioso: en SQLite, `'t' = 1` es falso, y con eso los motivos «dado de baja» y «no disponible» del semáforo quedaban muertos.

Protocolo sostenido: leer línea por línea, probar en copia o transacción reversible, respaldar antes de tocar estructura y declarar todo uso de IA.

CÓMO LEER LAS EVIDENCIAS

Cada sección cierra con los archivos que permiten reproducir lo afirmado. Las rutas son relativas al repositorio `02_Base-de-datos-II`, salvo las de la Unidad 7, que viven en `Proyecto_Food_Store_MongoDB`.

Del primer parcial a esta entrega

U	DÓNDE QUEDÓ EL PRIMER PARCIAL	QUÉ AGREGA ESTA ENTREGA, CON EVIDENCIA
4	Esquema heredado con redundancia y sin trazabilidad de cambios.	BCNF alcanzado con 0 diferencias contra las DF reales, migración reversible y reporte desnormalizado 319× más rápido.
5	Accesos resueltos con un usuario administrador compartido.	Seis roles con mínimo privilegio probados con `SET ROLE`, auditoría activa e incidente reconstruido desde el log.
6	Sin respaldo programado ni procedimiento de restauración.	`backup`, `restore` y chequeo de salud automatizados; simulacro de desastre con 48,3 s de RTO medido.
7	PostgreSQL como único motor del dominio.	Modelo documental con 10 pedidos migrados, consultas cruzadas equivalentes y criterio escrito de elección.
8	Sin cliente móvil ni estrategia de sincronización.	SQLite en la tablet, semáforo de cuatro reglas y seis casos de conflicto, incluida la prueba de que el móvil no escribe.

CONTENIDO

1 El proyecto y la arquitectura de datos 2
2 Normalización avanzada y redundancia · U4 3
3 Administración y seguridad · U5 5
4 Mantenimiento y recuperación · U6 7
5 NoSQL y almacenamiento documental · U7 8
6 Base móvil e integración · U8 10
7 Uso de IA: DUIA, bitácora y decisiones 12
8 Checklist de la entrega y observaciones 13
A Anexo · Repositorios y reproducibilidad 14

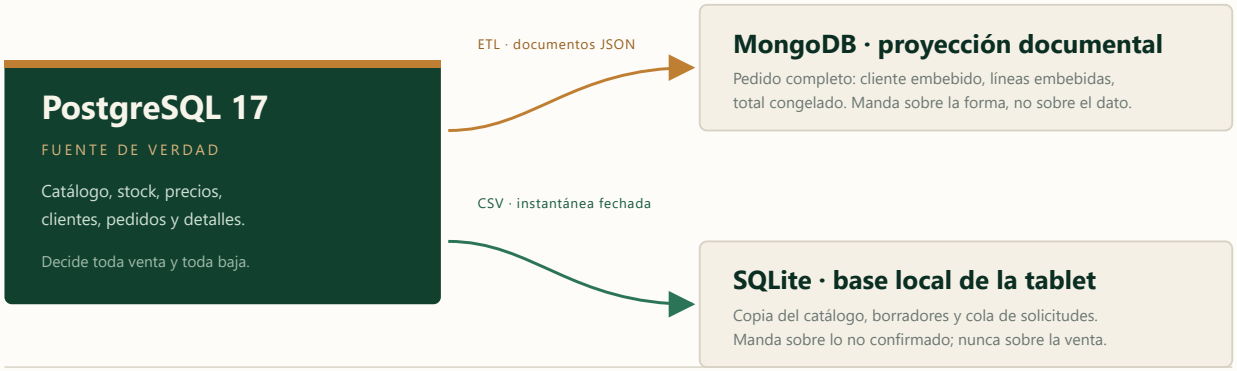
CAPÍTULO 1 · MARCO DE TRABAJO

01 El proyecto y la arquitectura de datos

Un dominio gastronómico, tres motores y una jerarquía explícita de autoridad.

«Food Store» es un sistema de gestión para un comercio gastronómico. La segunda entrega del TPI completa su capa de datos con tres tecnologías que no compiten: se reparten responsabilidades. Entender esa jerarquía es la condición para leer el resto del informe.

Un dominio, tres motores, una jerarquía



La tablet no tiene conexión, credenciales ni driver hacia PostgreSQL. El único canal es un archivo.

el móvil propone, el servidor dispone

MOTOR	QUÉ GUARDA	AUTORIDAD	QUÉ NO PUEDE HACER
PostgreSQL 17.10	Catálogo completo, 20.000 clientes, 200.000 pedidos y detalles, stock, precios, categorías, lotes y depósitos.	**Fuente de verdad.** Decide precio, stock, existencia y toda venta.	—
MongoDB base documental	Pedidos desnormalizados: un documento por pedido, con cliente y líneas embebidos y total congelado.	Manda sobre la **forma del documento**, no sobre el dato.	No origina ni corrige precios, stock o ventas. Si discrepa, gana PostgreSQL.
SQLite 3.50.4	Copia fechada del catálogo, borradores de pedido y cola de solicitudes de validación.	Manda sobre los **borradores sin confirmar**: son suyos y nadie más los ve.	No confirma ventas, no descuenta stock y no escribe en PostgreSQL.

El dataset de trabajo

Todas las mediciones se hicieron sobre una copia de trabajo (`Food_Store_Copia`), nunca sobre la base original. La carga masiva del TP3 dejó un volumen suficiente para que los planes de ejecución sean realistas y no de laboratorio vacío.

TABLA	FILAS	OBSERVACIÓN
categoria	10	Base del catálogo
cliente	20.000	Datos sintéticos
producto	50.000	49.744 vendibles
pedido	200.000	Un año de ventas
detalle_pedido	200.000	Una línea por pedido

Método: la ejecución como árbitro

El equipo sostuvo cuatro reglas operativas en las cinco unidades. No son declarativas: se ven en los scripts y en las bitácoras.

- **Copiar antes de tocar.** DDL y migraciones se ejecutan sobre una copia, con `pg_dump` previo.
- **Probar en reversible.** DML crítico dentro de `BEGIN … ROLLBACK`; si el resultado no se verifica, no se confirma.
- **Medir, no estimar.** Todo índice, vista o desnormalización se justifica con `EXPLAIN (ANALYZE, BUFFERS)`.
- **Declarar la IA.** Cada pieza generada tiene herramienta, prompt, verificación humana y decisión final.

> **TRAZABILIDAD**
> Repositorio relacional y móvil: `github.com/ismaSaleme12/02_Base-de-datos-II`.
> Repositorio documental: `github.com/ismaSaleme12/Proyecto_Food_Store_MongoDB_SqlLite`.

UNIDAD 4 · NORMALIZACIÓN AVANZADA Y REDUNDANCIA

02 De 3FN a BCNF, y una redundancia que se justifica

Refactorización heredada, migración sin pérdida y desnormalización controlada con salvaguardas.

La unidad se divide en dos decisiones opuestas y complementarias: **normalizar lo que estaba mal** y **desnormalizar lo que el reporte necesita**. La primera se demuestra con dependencias funcionales; la segunda, con una medición antes y después y una auditoría que prueba que la copia no se desincronizó.

2.1 · El esquema heredado y sus dependencias funcionales

El caso de partida es una relación que registra qué responsable controla cada lote en cada depósito: `control_lote_almacen(lote_id, deposito_id, responsable_control_id)`, con clave primaria compuesta `(lote_id, deposito_id)`. La regla de negocio agrega un dato maestro que la tabla no modela bien: **cada responsable pertenece a un único depósito**.

DEPENDENCIA FUNCIONAL	¿DETERMINANTE ES SUPERCLAVE?	ORIGEN
`{lote_id, deposito_id} → responsable_control_id`	**Sí** (PK compuesta)	Un lote y un depósito determinan quién controla.
`responsable_control_id → deposito_id`	**No** — viola BCNF	Cada responsable pertenece a un único depósito (dato maestro de personal).
`{lote_id, responsable_control_id} → deposito_id`	Sí (clave candidata)	Surge de la clausura; no es una DF independiente.

2.2 · Por qué 3FN no alcanza

Los tres atributos son primos: cada uno pertenece a alguna clave candidata. Por eso toda dependencia funcional tiene consecuente primo y el esquema **sí cumple 3FN**. Pero la definición de BCNF es más exigente: *todo determinante debe ser superclave*. Y `responsable_control_id` no lo es: su clausura agrega `deposito_id`, pero no `lote_id`.

CONJUNTO	CLAUSURA	¿SUPERCLAVE?
{lote_id}	{lote_id}	No
{deposito_id}	{deposito_id}	No
{responsable}	{responsable, deposito_id}	No
{lote_id, deposito_id}	los tres atributos	**Sí**
{lote_id, responsable}	los tres atributos	**Sí**

CONCLUSIÓN FORMAL

`control_lote_almacen` está en 3FN pero **no en BCNF**. La dependencia violatoria es `responsable → depósito`; la descomposición es obligatoria, no una preferencia de diseño.

2.3 · Las anomalías, con nombres y apellidos

Inserción

Un responsable nuevo (803) asignado al depósito 30, todavía sin lotes, no se puede registrar: la PK y los `NOT NULL` obligan a inventar un lote ficticio.

Borrado

La fila `(503, 31, 802)` es la única que vincula al responsable 802 con el depósito 31. Si se anula ese control, se pierde para siempre el dato maestro.

Actualización

«801 pertenece al depósito 30» está repetido en dos filas. Si 801 se traslada al 32 hay que actualizar N filas; una actualización parcial deja la base inconsistente.

2.4 · Descomposición sin pérdida y migración verificada

Se aplica el algoritmo sobre la dependencia violatoria: `R1 = X ∪ Y` y `R2 = R − Y`. El atributo común es `responsable_control_id`, que es clave primaria —y por lo tanto superclave— de `responsable_deposito`. Cada fila de `control_lote` se combina con exactamente una fila de R1: la reunión no puede perder tuplas ni inventar tuplas espurias.

```
-- R1: el dato maestro del responsable
CREATE TABLE responsable_deposito (
  responsable_control_id BIGINT PRIMARY KEY
        REFERENCES usuario(id),
  deposito_id BIGINT NOT NULL REFERENCES deposito(id)
);

-- R2: el hecho "quién controla qué lote"
CREATE TABLE control_lote (
  lote_id BIGINT NOT NULL REFERENCES lote(id),
  responsable_control_id BIGINT NOT NULL
        REFERENCES responsable_deposito(responsable_control_id),
  PRIMARY KEY (lote_id, responsable_control_id)
);

-- Compatibilidad: reconstruye la relación original
CREATE VIEW v_control_lote_almacen AS
SELECT cl.lote_id, rd.deposito_id, cl.responsable_control_id
FROM control_lote cl
JOIN responsable_deposito rd
  ON rd.responsable_control_id = cl.responsable_control_id;
```

La migración es idempotente: `TRUNCATE` y recarga desde la tabla original, sin pérdida. Se ejecutó sobre `Food_Store_Copia` y se verificó con doble `EXCEPT`.

CONTROL	RESULTADO
Filas del original ausentes en la vista	**0**
Filas de la vista ausentes en el original	**0**
Conteos: original / vista / R1 / R2	3 / 3 / 2 / 3

PLAN DE REVERSIÓN

La vista de compatibilidad permite que las aplicaciones sigan leyendo la relación original. Para volver atrás alcanza con recrear `control_lote_almacen` desde la vista y eliminar R1/R2; la migración no borra el original hasta que la verificación da cero filas.

2.5 · Desnormalización controlada: el reporte que no podía esperar

El reporte «top 5 de categorías por ventas del día» tardaba **73,935 ms**. El costo no estaba en el `GROUP BY`, sino en dos `Parallel Seq Scan`: el filtro `fecha::date = MAX(fecha)` no es indexable y descarta 199.596 de 200.000 filas. Se eligió una **tabla de agregados diarios** mantenida por disparador, no una vista materializada: el panel se pide en tiempo real y una materializada quedaría obsoleta entre refrescos.

73,9 ms
ANTES
NORMALIZADA

0,23 ms
DESPUÉS
DESNORMALIZADA

ESTRUCTURA	CLAVE / CONTROL
`resumen_venta_categoria_dia`	PK `(dia, categoria_id)`; FK a categoría; `CHECK total_vendido >= 0`
Backfill histórico	3.660 filas (366 días × 10 categorías)
Sincronización	Trigger `AFTER INSERT/UPDATE/DELETE` por fila sobre `detalle_pedido`; aplica deltas
Auditoría de sincronía	`FULL JOIN` contra la fuente; **0 filas** de diferencia

SALVAGUARDAS DE LA REDUNDANCIA

- La fuente de verdad no se toca: la tabla es derivada y reconstruible.
- El disparador cubre alta, baja y modificación, no sólo inserción.
- La consulta de auditoría detecta cualquier desvío (`IS DISTINCT FROM`).
- Reversión trivial: `DROP TABLE` + `DROP TRIGGER` , sin pérdida de información.
- Se probó un `INSERT` de 2 × 1868,53: el resumen subió exactamente 3.737,06 y el `ROLLBACK` dejó todo intacto.

Por qué acá no se usa una materializada. En la Unidad 3 sí se usó `vm_facturacion_categoria_mes` (523 ms → 0,82 ms) porque el reporte era histórico mensual y toleraba staleness. La decisión cambia con el requisito: mismo dominio, distinta latencia aceptable.

DECISIÓN REGISTRADA · U4

Se refactorizó `control_lote_almacen` hasta BCNF porque la redundancia `responsable → depósito` generaba anomalías de inserción, borrado y actualización. Se desnormalizó `resumen_venta_categoria_dia` porque el reporte tenía un costo dominado por recorridos completos y un requisito de frescura que una materializada no cumplía. **Una se normalizó por corrección; la otra se desnormalizó por rendimiento, con auditoría y reversión.**

2.6 · La migración como procedimiento reversible

La migración no fue un `ALTER TABLE` suelto: se ejecutó como un procedimiento con cuatro pasos y una condición de corte. Si la verificación no daba cero, no se confirmaba.

PASO	ACCIÓN	CONDICIÓN DE AVANCE
0	`pg_dump` de la copia de trabajo.	Archivo generado y restaurable.
1	Crear `responsable_deposito` y `control_lote` sin tocar la tabla original.	PK y FK creadas.
2	Cargar R1 con `SELECT DISTINCT` y R2 con los hechos.	Conteos 3 / 2 / 3.
3	Doble `EXCEPT` contra la relación original.	**0 y 0**; recién ahí se confirma.

PLAN DE REVERSIÓN, EN DOS NIVELES

Dentro de la transacción: DDL y carga van en el mismo `BEGIN … COMMIT` . Si la verificación falla, `ROLLBACK` y el esquema queda como estaba.

Fuera de la transacción: la tabla original no se elimina hasta que la auditoría da cero. La vista `v_control_lote_almacen` reconstruye la relación heredada para las aplicaciones que todavía la leen.

LA DECISIÓN QUE HACE REVERSIBLE LA MIGRACIÓN

No borrar el original. Una migración que destruye la fuente antes de verificar no se puede revertir: sólo se puede lamentar. El orden es *crear, cargar, comparar y recién después retirar*.

Evidencias U4: `db/tp_fnbc_control_lote.sql` · `db/tp_desnormalizacion_top_categorias.sql` · `docs/Informe_TP_Unidad4_FNBC_Desnormalizacion.pdf` · `docs/normalizacion.md` .

UNIDAD 5 · ADMINISTRACIÓN Y SEGURIDAD

03 Mínimo privilegio, auditoría y reconstrucción de un incidente

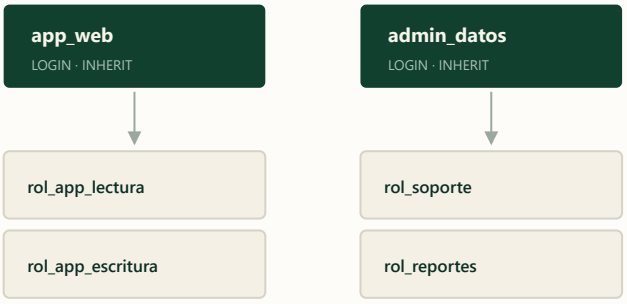
Seis roles, permisos por columna, un simulacro de acceso indebido y una decisión humana sobre cada hallazgo.

La seguridad no se declara: se verifica. Esta unidad diseñó un esquema de roles, lo probó con `SET ROLE` real, activó auditoría, reconstruyó un ataque a partir del log y anonimizó los datos antes de enviarlos a la IA. El hilo conductor es el principio de mínimo privilegio: **cada rol recibe sólo lo que necesita y nada más**.

3.1 · Modelo de roles: grupos sin login, dos logins sin superpoderes

Cuatro roles de grupo (`NOLOGIN`) agrupan capacidades y dos roles de login las heredan. Ninguno es `SUPERUSER` , `CREATEDB` ni `CREATEROLE` . El mantenimiento se hace como `postgres` , no desde la aplicación.

ROL	TIPO	QUÉ PUEDE HACER
`rol_app_lectura`	Grupo	Leer vistas y columnas no sensibles de `usuario` .
`rol_app_escritura`	Grupo	Sólo `EXECUTE` : crear pedido y autenticar. Sin acceso a tablas base.
`rol_soporte`	Grupo	Diagnóstico: `SELECT` sobre tablas operativas, sin `password_hash` .
`rol_reportes`	Grupo	Vista materializada de facturación y columnas agregables.
`app_web`	Login	Hereda lectura + escritura. No lee tablas base.
`admin_datos`	Login	Hereda soporte y ejecuta procedimientos de administración.



Ningún rol de login es SUPERUSER, CREATEDB ni CREATEROLE.

PERMISOS POR COLUMNA

`password_hash` no se otorga a ningún rol de grupo. La lectura de `usuario` se hace con `GRANT SELECT (columnas)` , no con `SELECT *` .

3.2 · Verificación: lo que se probó y lo que dio

Tabla base

`SET ROLE app_web; SELECT * FROM cliente;`

PERMISO DENEGADO

Vista permitida

`SET ROLE app_web; SELECT * FROM v_productos_vigentes…;`

1 FILA

Escalamiento

`SET ROLE app_web; GRANT admin_datos TO app_web;`

DENEGADO

HALLAZGO REAL · EL REVOKE QUE NO REVOCABA

La primera versión hacía `GRANT SELECT ON usuario` y luego `REVOKE SELECT (password_hash)` . La verificación mostró que `has_column_privilege` seguía en `true` : en PostgreSQL, un `REVOKE` por columna **no recorta** un `GRANT` a nivel tabla. Se corrigió a `REVOKE ALL` + `GRANT` explícito por columnas. El mismo error se detectó en el code review de la Unidad 8 (H2).

3.3 · Auditoría de accesos y operaciones

PARÁMETRO	VALOR	EFECTO
`log_connections`	on	Registra cada intento de conexión.
`log_disconnections`	on	Permite parear sesión y duración.
`log_statement`	mod	INSERT/UPDATE/DELETE + DDL; no registra SELECT puros.
`log_line_prefix`	`%m [%p] %u@%d %h %a`	Timestamp, PID, usuario, base, host y aplicación.
`logging_collector`	on	Captura stderr en archivos.

La prueba ejecutó una lectura, una inserción, una actualización y un login fallido. El `FATAL: no existe el rol «usuario_inexistente»` quedó capturado. El `SELECT` no aparece con `mod` : es el control negativo que demuestra que el nivel elegido es el correcto.

QUÉ RESPONDE EL LOG —Y QUÉ NO

Responde **quién** (usuario y rol), **qué** (sentencia), **cuándo** (timestamp con milisegundos) y **desde dónde** (IP y puerto). El PID permite reconstruir la secuencia conexión → operación → desconexión.

No responde el detalle fila por fila (valores `OLD/NEW`), la cantidad de filas afectadas ni el usuario de aplicación detrás de un pooler. Para eso se necesitaría `pgaudit` .

EL LOG TAMBIÉN ES UN ACTIVO SENSIBLE

Contiene correos, patrones de acceso, IPs y horarios. Debe leerlo sólo el administrador de la instancia, con permisos de sistema restringidos y sin `GRANT` general en el motor.

3.4 · Simulacro de acceso indebido y reconstrucción forense

Se ejecutó una secuencia controlada sobre un usuario víctima sintético: tres llamadas fallidas a `fn_autenticar` , una exitosa, una lectura masiva de `usuario` y un intento de escalamiento a `admin_datos` . El log se anonimizó antes de entregarlo a la IA.

La hipótesis reconstruida fue **compromiso de credencial** y escalamiento horizontal → vertical. La IA acertó la secuencia y el evento más grave, pero **omitió** que `fn_autenticar` devuelve un booleano y no genera `FATAL` —el conteo de fallidos debe hacerse sobre resultados, no sobre conexiones— y afirmó una «madrugada sospechosa» que el log anonimizado no permite sostener. Esa inferencia se descartó.

CONTENCIÓN DECIDIDA POR EL EQUIPO

1. Bloqueo temporal del origen sólo si se confirma reincidencia desde la misma IP real.
2. Rotación inmediata de la clave de la víctima y de `app_web` / `admin_datos` , con vault.
3. Verificación de `pg_auth_members` para confirmar que no hubo membresía otorgada.
4. Endurecimiento: `log_statement='mod'` permanente y alerta ante más de 5 `FALSE` por minuto.
5. Nada sugerido por la IA se ejecuta sin `BEGIN … ROLLBACK` y respaldo previo.

La constancia de escalamiento es verificable: `SET ROLE app_web; GRANT admin_datos TO app_web;` falla con «se ha denegado el permiso para otorgar el rol».

HALLAZGO METODOLÓGICO

La primera versión probaba el escalamiento con `DO … EXECUTE 'GRANT admin_datos'`. Al correr como superusuario, el `GRANT` **tuvo éxito** y dejó a `app_web` dentro de `admin_datos`. Se revocó y se reescribió con `SET ROLE app_web`. Lección: una prueba de permisos se corre con el rol atacado, nunca como superusuario.

3.5 · Anonimización y auditoría de seguridad asistida por IA

Antes de enviar cualquier dato a la IA se construyó `usuario_anon` con seudónimos deterministas (`usuario.NNNNN@sintetico.test`) y meses en lugar de fechas exactas. La agregación por rol y mes sobre la tabla anonimizada y sobre la real devolvió **7 grupos idénticos** y un `EXCEPT` cruzado de **0 filas**.

La auditoría de seguridad del esquema también se asistió con IA. La IA propuso `GRANT ALL ON ALL TABLES` a `rol_soporte` y `app_web` «para simplificar», y `SUPERUSER` a `admin_datos` «para mantenimiento». Ambas propuestas se rechazaron: violaban mínimo privilegio y exponían `password_hash`.

PROPUESTA DE IA	DECISIÓN HUMANA	MOTIVO
`GRANT ALL` a soporte y app	RECHAZADA	Exponía `password_hash` y daba escritura innecesaria.
`SUPERUSER` a `admin_datos`	RECHAZADA	El mantenimiento se hace como `postgres`.
`ALTER DEFAULT PRIVILEGES`	ACEPTADA	Las tablas futuras heredan la política.
Reconstruir `usuario` y `fn_autenticar`	ACEPTADA	El esquema heredado no tenía credenciales.

DECISIÓN FINAL HUMANA

El esquema quedó con mínimo privilegio, permisos por columna y verificación real. La IA propuso, el equipo ejecutó, midió y decidió. La aprobación final no se delegó.

3.6 · Reconstrucción forense: cinco eventos, una sola sesión

#	EVENTO EN EL LOG	LECTURA
1	Tres llamadas a `fn_autenticar` con resultado `FALSE`.	Fuerza bruta de baja intensidad o *credential stuffing* manual. No genera `FATAL` porque la función retorna un booleano.
2	Una llamada con resultado `TRUE`.	La credencial cedió. El conteo de intentos fallidos debe hacerse sobre resultados, no sobre conexiones.
3	`SELECT` masivo sobre `usuario`, mismo PID.	Lectura de datos personales. Sin `SET ROLE`, el log no prueba qué columnas habría visto `app_web`.
4	`GRANT admin_datos TO app_web` → `permission denied`.	El diseño de roles contuvo el escalamiento. Es el evento más grave y el único que falló.
5	Sin `FATAL` y sin conexión nueva.	Todo ocurrió en la misma sesión: el patrón es horizontal (misma altura) y después vertical (más privilegio).

LO QUE EL LOG NO DICE

No dice qué filas se leyeron, ni el valor anterior y nuevo de cada una, ni el usuario de aplicación detrás de un pooler. Tampoco conserva la zona horaria ni la reputación de la IP. Por eso se descartó la «madrugada sospechosa» que infería la IA: **el log no la sostiene**.

La atribución se apoya en el `PID` y en el `log_line_prefix` `%m [%p] %u@%d %h %a`. Ese prefijo permite parear conexión, operación y desconexión sin adivinar. Para el detalle fila por fila haría falta `pgaudit`, que quedó fuera del alcance declarado.

Evidencias U5: `db/roles.sql` · `db/auditoria_test.sql` · `db/simulacro_incidente.sql` · `docs/verificacion_roles.txt` · `docs/log_auditoria_prueba.txt` · `docs/informe_incidente.md` · `docs/log_simulacro_anonimizado.txt` · `docs/verificacion_equivalencia.md` · `BITACORA.md`.

UNIDAD 6 · MANTENIMIENTO DE UNA BASE DE DATOS

04 Un respaldo que nunca se restauró es sólo una carpeta

Plan programado, automatización con OpenCode y simulacro de desastre contra reloj.

El mantenimiento se demuestra de dos maneras: con un plan que se ejecuta y con una restauración que se prueba. La unidad automatizó respaldo, control de salud y revisión de mantenimiento, y después hizo lo que casi nunca se hace: **restaurar el respaldo en una base distinta y medir cuánto tarda**.

4.1 · El plan: cuatro tareas, cuatro evidencias

TAREA	FRECUENCIA	EVIDENCIA / SALIDA	CRITERIO DE ÉXITO
Respaldo de `Food_Store_Copia`	Diaria	`.dump` en formato custom (`-Fc`) + log en `logs/`.	`pg_dump` con código 0 y archivo mayor a 0 bytes.
Chequeo de salud	Diaria	Salida de `health_check.sql` : tamaño, conexiones, tuplas muertas, consultas activas y ratio de cache.	Métricas operativas normales, sin bloqueos prolongados.
Revisión de mantenimiento	Semanal	Estado de `autovacuum` / `autoanalyze` y salida de `maintenance.sql` .	Estadísticas actualizadas y tuplas muertas controladas, sin `VACUUM FULL` periódico.
Restauración de prueba	Semanal	Base `foodstore_restore_test` validada con conteos y restricciones lógicas.	Restauración sin errores y conteos coincidentes con la base viva.

4.2 · Automatización: dos scripts, una rutina

`backup_foodstore.ps1` genera el respaldo con timestamp, escribe el log, propaga el código de salida y **no contiene contraseñas en texto plano**: usa `$env:PGPASSWORD` en tiempo de ejecución. `restore_test.ps1` selecciona el `.dump` más reciente, elimina la base de prueba si existe, la crea, restaura con `pg_restore --no-owner --no-privileges` y ejecuta las validaciones con un cronómetro alrededor de todo el procedimiento.

La restauración **nunca** apunta a la base principal: se hace sobre `foodstore_restore_test`. Esa separación es parte del protocolo, no un detalle de comodidad.

```
# backup_foodstore.ps1 — núcleo
$Timestamp = Get-Date -Format "yyyyMMdd_HHmmss"
$DumpFile  = Join-Path $BackupDir "foodstore_$Timestamp.dump"

& $PgDumpPath -h $DbHost -p $DbPort -U $DbUser `
    -d $DbName -Fc -f $DumpFile

# restore_test.ps1 — restauración y RTO
$Stopwatch = [Diagnostics.Stopwatch]::StartNew()
& $PgRestorePath -d $TestDbName --no-owner `
    --no-privileges $DumpPath
# ... validaciones de conteo y restricciones ...
$Stopwatch.Stop()
```

4.3 · El simulacro, en números

~1,5 h
RPO OBSERVADO
ANTIGÜEDAD DEL RESPALDO

48,3 s
RTO OBSERVADO
OBJETIVO: < 300 S

18,4 MB
TAMAÑO DEL DUMP
FORMATO CUSTOM

0
PRODUCTOS Y DETALLES
INVÁLIDOS

El RTO se midió desde la creación de la base de prueba hasta la finalización de las validaciones. El RPO es la antigüedad del último respaldo al iniciar el simulacro. Ambos quedaron por debajo del objetivo declarado.

VALIDACIONES EJECUTADAS

Conteo de las cinco tablas contra la base viva, productos con precio o stock negativos (`0`) y detalles con cantidad no positiva o precio negativo (`0`). La restauración no se consideró exitosa por «no dar error», sino por **coincidir en los datos**.

4.4 · Informe post-incidente: el problema real y la mejora

El problema no fue el desastre simulado, sino la autenticación: la directiva `scram-sha-256` de PostgreSQL 16/17 exigió manejar credenciales sin exponerlas en el script. Se resolvió con `$env:PGPASSWORD` en tiempo de ejecución. La mejora propuesta es concreta: **retención de los últimos 7 respaldos diarios** y compresión de los antiguos con gzip/tar en almacenamiento secundario, para reducir espacio y agilizar la selección del respaldo óptimo.

QUÉ PASA SI SE RESTAURA UN RESPALDO VIEJO

Es la situación más peligrosa: la tablet tiene una instantánea con `fecha_copia` posterior al punto de restauración y **cree saber más que el servidor**. Un producto dado de baja puede reaparecer activo y un borrador puede compararse contra precios que ya no existen.

La respuesta es editorial: el servidor publica una instantánea nueva con `fecha_copia` posterior al restore y rechaza solicitudes de épocas anteriores. La tablet recarga la cache completa, conserva los borradores y los revalida: lo que estaba `SIMULADO_APROBADO` vuelve a `PENDIENTE`.

4.5 · Rutina operativa y responsables

Salud diaria

Tamaño de la base, conexiones activas, ratio de cache, tuplas muertas y consultas en curso. Se detecta degradación antes de que sea incidente.

Mantenimiento semanal

`ANALYZE` de las tablas críticas y `VACUUM` estándar. Nunca `VACUUM FULL` periódico: bloquea y reescribe el archivo.

Conflictos y rechazos

El `CONFLICTO` lo resuelve caja en el momento; el `RECHAZADO` se revisa al cierre. Un producto que se rechaza siempre es una señal de stock mal cargado.

Evidencias U6: `SPEC_U6.md` · `backups/backup_foodstore.ps1` · `backups/restore_test.ps1` · `db/health_check.sql` · `db/maintenance.sql` · `docs/informe_u6.pdf` · `logs/`.

UNIDAD 7 · NOSQL Y NUEVAS FORMAS DE ALMACENAMIENTO

05 El pedido como documento: MongoDB frente a PostgreSQL

Modelado documental, migración real y una comparación que no se queda en la teoría.

La pregunta de la unidad no es «¿MongoDB o PostgreSQL?», sino **«¿qué parte del dominio gana siendo documento y qué parte pierde?»**. Se migraron 10 pedidos reales, se ejecutaron las mismas consultas en ambos motores y se compararon resultados e identificadores. La respuesta es matizada: el documento gana en la lectura del pedido completo; PostgreSQL sigue siendo el sistema de registro.

5.1 · Un documento = un pedido completo

El subconjunto elegido involucra cinco tablas: `pedido`, `detalle_pedido`, `cliente`, `producto` y `categoria`. La decisión estructural es embeber las líneas y los datos del cliente en un único documento. Así, la escritura del pedido completo es **atómica por diseño**: un solo documento, sin transacción multi-documento.

```
{
  _id: 1,                    // = pedido.id original
  pedido_id: 1,
  fecha: ISODate("2026-06-28T23:06:30Z"),
  forma_pago: "EFECTIVO",
  cliente: {
    cliente_id: 453, nombre: "Cliente 453",
    email: "cliente453@example.com",
    telefono: "555-000453"
  },
  lineas: [{
    producto_id: 1, nombre: "Producto 1",
    categoria: "Bebidas", cantidad: 5,
    precio_unitario: 1868.53, subtotal: 9342.65
  }],
  total: 9342.65,            // congelado al vender
  origen: { motor: "PostgreSQL", base: "practica_bd2_tp" }
}
```

QUÉ SE EMBEBIÓ Y QUÉ QUEDÓ AFUERA

ELEMENTO	DECISIÓN	MOTIVO
`lineas[]`	Embebido	Entidad débil: no existe una línea sin pedido y se lee siempre con la cabecera.
`cliente{}`	Embebido parcial	Sólo lo que se muestra al leer el pedido; se conserva `cliente_id` para volver.
`producto`	Embebido histórico	Nombre y categoría del momento; el catálogo mutable queda fuera.
Precio actual / stock	Fuera	Dato compartido por miles de pedidos: duplicarlo generaría inconsistencia.
`total`	Precalculado	Evita `$sum` en cada lectura; se congela al vender.

TRAZABILIDAD

`_id` es el `pedido.id` original: no se inventan claves. `lineas[].precio_unitario` es el precio histórico de `detalle_pedido`, nunca el precio actual del producto.

5.2 · Migración y reglas de integridad

La extracción se hizo con `jsonb_agg` + `jsonb_build_object`: una consulta arma cada pedido completo como JSON, agrega las líneas y calcula el total. La carga se hizo con `insertMany()`; la alternativa equivalente es volcar a `datos_migracion.json` y usar `mongoimport --jsonArray`.

Se migraron **10 pedidos** (IDs 1–7, 50001, 100001 y 150001), elegidos para cubrir las tres formas de pago y para que cuatro compartan el producto 1, de modo que la consulta por líneas devuelva más de una fila.

DETALLE DE ZONA HORARIA

PostgreSQL guarda `TIMESTAMPTZ` con offset `-03`; MongoDB lo normaliza a UTC. El pedido 50001 queda como `2026-05-04T00:32:56Z`. Es el mismo instante: cambia la representación, no el dato.

La Regla 1 (sólo productos activos) se conserva en la extracción; en runtime no es exigible porque `producto` vive en otra colección. La Regla 2 (todo pedido tiene al menos una línea) pasa a ser **estructural**: `lineas` es un arreglo con `minItems: 1` en el validador `$jsonSchema`.

CONTROL	IMPLEMENTACIÓN	RESULTADO
Campos obligatorios y tipos	`$jsonSchema`	Activo
Dominio de `forma_pago`	`enum`	Activo
`cantidad ≥ 1`, `precio ≥ 0`	`minimum`	Activo
Pedido sin líneas	`minItems: 1`	RECHAZADO
Línea con `cantidad: 0`	`minimum: 1`	RECHAZADO
Producto inexistente o inactivo	Sin FK entre colecciones	NO EXIGIBLE

5.3 · Las cuatro consultas y su verificación cruzada

CONSULTA	MONGODB	FILAS	RESULTADO / IDS
a) Todos los pedidos	`find({}, {_id, fecha, forma_pago, total})`	10	Ordenados por `_id`
b) Forma de pago `TARJETA`	`find({forma_pago:"TARJETA"})`	4	`{2, 3, 5, 50001}`
c) Contiene el producto 1	`find({"lineas.producto_id": 1})`	4	`{1, 50001, 100001, 150001}`
d) Total > $10.000	`find({total: {$gt: 10000}}).sort({total:-1})`	3	`{2, 3, 7}`

COINCIDENCIA CON POSTGRESQL

CONSULTA	POSTGRESQL	MONGODB	¿MISMOS IDS?
Forma de pago `TARJETA`	4	4	SÍ
Producto 1 en el detalle	4	4	SÍ

La comparación se restringe a los 10 pedidos migrados. Comparar contra los 200.000 pedidos de la base no sería equivalente: la consigna pide el mismo conjunto de datos en ambos motores.

EL UMBRAL DE LA CONSULTA D), ELEGIDO CON DATOS

Sobre los 200.000 pedidos, el total promedio es **$8.233,91**, la mediana **$6.774,93** y el p90 **$17.210,54**. Un corte en $10.000 deja por encima al **31,6 %** (63.178 pedidos): el tercio de mayor valor. En la muestra selecciona 3 de 10, una proporción consistente.

UN HALLAZGO SOBRE EL DATASET

En la base real, `pedido` y `detalle_pedido` tienen 200.000 filas cada una: la carga masiva generó **una línea por pedido**, de modo que la relación N:M quedó degenerada en 1:1. El modelo documental soporta N líneas, pero conviene decirlo: las conclusiones sobre «muchas líneas por pedido» no están respaldadas por los datos actuales.

5.4 · La decisión tecnológica: PostgreSQL como registro, MongoDB como lectura

Integridad referencial

Las líneas dependen de `producto` y `cliente`. PostgreSQL lo garantiza con FK; MongoDB no puede imponerlas entre colecciones. La Regla 1 no se sostiene desde el motor.

Patrón de acceso

El documento gana al leer el pedido completo: una lectura contra un JOIN de cuatro tablas. Pero el catálogo, el stock y los reportes agregados siguen siendo naturalmente relacionales.

Volumen y escritura

200.000 pedidos con una línea cada uno no justifican cambiar de motor. El modelo documental quedó degenerado por la carga, no por el diseño.

CONCLUSIÓN DE LA UNIDAD

Food Store debe **mantener PostgreSQL como sistema de registro** y evaluar MongoDB únicamente como **réplica de lectura** del pedido completo, alimentada por el mismo proceso que hoy genera los documentos. La combinación razonable no es reemplazar: es especializar.

5.5 · Contraste de afirmaciones asistido por IA

AFIRMACIÓN	VEREDICTO	CONTRASTE
«MongoDB no soporta ACID y no garantiza que pedido y líneas se guarden juntos».	FALSA	Soporta transacciones multi-documento desde 4.0 (replica sets) y 4.2 (sharded). Además, al embeber las líneas, pedido + líneas es **un solo documento**: la atomicidad es estructural.
«MongoDB no valida esquema ni restricciones, así que se pierden los CHECK».	PARCIAL	`$jsonSchema` con `validationAction: "error"` replica required, tipos, enum y rangos. **Donde acierta** es en la integridad referencial: no hay FK entre colecciones.

5.6 · Lo que MongoDB sí puede exigir — y lo que no

REGLA	IMPLEMENTACIÓN	EFECTO
Campos y tipos	`$jsonSchema` con `required` y `bsonType`	Rechaza documentos incompletos.
Dominio de `forma_pago`	`enum` de tres valores	Rechaza valores fuera del dominio.
`cantidad ≥ 1`, `precio ≥ 0`	`minimum`	Rechaza líneas inválidas.
Pedido sin líneas	`minItems: 1`	La Regla 2 pasa a ser estructural.
Integridad referencial	No hay FK entre colecciones	NO EXIGIBLE

El validador corre con `validationLevel: "strict"` y `validationAction: "error"`: no advierte, rechaza la escritura.

TRES ÍNDICES, TRES CONSULTAS

Los índices no se crearon «por las dudas»: cada uno responde a una de las consultas comparadas con PostgreSQL.

`lineas.producto_id`	Consulta c: pedidos que contienen el producto 1.
`forma_pago`	Consulta b: pedidos con tarjeta.
`total: -1`	Consulta d: total mayor a $10.000, ordenado.

EL LÍMITE QUE EL ÍNDICE NO RESUELVE

Un índice acelera la lectura, no reemplaza una FK. Que un documento apunte a un producto inexistente sigue siendo posible: por eso la Regla 1 se conserva en la extracción y no en runtime.

Evidencias U7: `consultas_mongodb.js` · `consultas_postgresql.sql` · `datos_migracion.json` · `salida_mongodb.txt` · `salida_postgresql.txt` · `informe_u7_nosql.pdf`.

UNIDAD 8 · BASES DE DATOS PARA DISPOSITIVOS MÓVILES E INTEGRACIÓN

06 La tablet propone, el servidor dispone

Cuatro tablas SQLite, un semáforo de cinco estados y la prueba de que PostgreSQL no se movió.

El food truck vende en una feria sin señal. La tablet guarda una copia fechada del catálogo y arma borradores; cuando recupera conexión, **envía propuestas, no ventas**. La unidad demuestra esa jerarquía con un esquema local, un algoritmo de sincronización y una comparación de huellas antes y después. El resultado más importante no es un estado en verde: es que el servidor quedó idéntico.

6.1 · Tres capas y un solo sentido

1 ORIGEN	2 TRANSPORTE	3 ATERRIZAJE	4 TABLET
PostgreSQL Vista `v_snapshot_movil` de sólo lectura. Precio en centavos, dos booleanos traducidos y `fecha_copia` sellada en origen.	**CSV unidireccional** `\copy` de 50.000 filas. Sin driver, sin credenciales y sin canal de vuelta hacia el servidor.	**snapshot_servidor** Tabla no `STRICT` que recibe texto crudo. Una sola frontera de normalización: `'t'/'f'` → `1/0`.	**SQLite** UPSERT a `producto_cache`, borradores locales y cola de solicitudes. El semáforo sólo escribe en `cola_sync`.

La vista de exportación no es un `SELECT *`: traduce el contrato del servidor al de la tablet. `precio * 100` se convierte a entero para no arrastrar punto flotante al dinero; `disponible` condensa `activo AND stock > 0`; `eliminado` marca la baja lógica **sin filtrar la fila**.

La tabla de aterrizaje existe por un motivo concreto: `psql` exporta los booleanos como `t` / `f` y una tabla `STRICT` los rechaza en una columna `INTEGER`. Normalizar al entrar —y no en cada comparación del semáforo— deja una sola frontera de conversión.

CONTROL DE LA INSTANTÁNEA	RESULTADO
Productos exportados	**50.000**
Vendibles en la tablet	49.744
Dados de baja	0
Inconsistentes (`disponible AND eliminado`)	**0**
Filas en `producto_cache`	50.000

UNA FOTO, NO UNA VERDAD

`stock_visto` es una **observación**, no una reserva. La tablet no descuenta nada: si el stock cambió, el semáforo lo detecta y el servidor decide.

6.2 · Cuatro tablas, cuatro decisiones que no son obvias

TABLA	QUÉ GUARDA	DECISIÓN DE DISEÑO Y POR QUÉ
`producto_cache`	Copia de sólo lectura del catálogo: precio en centavos, `stock_visto`, `disponible`, `eliminado` y `fecha_copia`.	La fila de un producto dado de baja **no se borra**: si se borrara, los borradores históricos perderían su clave foránea y no se podría distinguir «dado de baja» de «nunca existió».
`pedido_borrador`	Cabecera del pedido armado en la tablet: UUID local, vendedor sintético, forma de pago y fecha.	`id_local` es un UUID con `CHECK GLOB`. `usuario` es un identificador de caja (`caja-01`), nunca un dato personal.
`item_borrador`	Renglones: producto, cantidad y `precio_centavos_visto`.	PK compuesta `(id_local, producto_id)`: un producto no se repite en el mismo borrador. FK a la cache con `ON DELETE RESTRICT`, para que una recarga no borre renglones en silencio.
`cola_sync`	Bandeja de salida: una solicitud de validación por borrador, con estado y motivo.	`solicitud_uuid` es PK; `id_local` **no** es único, porque C2 necesita dos solicitudes sobre el mismo borrador. Índice por `estado` porque el semáforo filtra siempre `PENDIENTE`.

LAS CUATRO TABLAS SON `STRICT`

SQLite no valida tipos por defecto: una columna `INTEGER` acepta `'hola'`. `STRICT` convierte esa tolerancia en error. Sobre ese piso se apoyan **17 restricciones** probadas una por una: precio negativo, cantidad cero, forma de pago fuera del dominio, UUID mal formado, FK rota y estado fuera de los cuatro valores.

LO QUE LA TABLET NO TIENE, A PROPÓSITO

No hay tabla de ventas, ni `cliente`, ni `password_hash`, ni conexión a PostgreSQL. Un `SIMULADO_APROBADO` es una propuesta verificada, **no una venta confirmada**. La ausencia es parte del diseño, no una omisión.

6.3 · El semáforo: el rojo gana siempre

El algoritmo evalúa cada ítem contra la instantánea y clasifica la solicitud completa. Los cuatro motivos de rojo son **excluyentes y ordenados**: el primero que aplica es el que ve el vendedor. El amarillo sólo aparece si no hay ningún rojo.

1. **No existe** en la instantánea.
2. **Dado de baja** (`eliminado = 1`).
3. **No disponible** (activo pero sin stock).
4. **No alcanza el stock** para la cantidad pedida.
5. **Precio distinto** al que vio el cliente → amarillo.

```
-- semaforo.sql · precedencia en un solo CASE
CASE
  WHEN s.id IS NULL      THEN 'no existe'
  WHEN s.eliminado = 1   THEN 'dado de baja'
  WHEN s.disponible = 0  THEN 'no disponible'
  WHEN i.cantidad > s.stock_visto
                         THEN 'no alcanza el stock'
  ELSE NULL
END AS motivo_rojo,
CASE
  WHEN i.precio_centavos_visto <> s.precio_centavos
       THEN 'precio cambió'
END AS motivo_amarillo
```

El `UPDATE` final filtra `estado = 'PENDIENTE'`. Correrlo dos veces no cambia nada: una solicitud resuelta no se reevalúa. Esa es la idempotencia del semáforo, distinta de la idempotencia del envío.

EL BUG QUE OBLIGÓ A REORDENAR LOS CASE

La primera versión consultaba «sin stock» antes que «dado de baja». Un producto discontinuado aparecía como un problema de cantidad, y el vendedor recibía una instrucción equivocada. El orden de los `WHEN` no es estilo: **es la regla de negocio**.

6.4 · Los seis casos, con la salida real

CASO	QUÉ SE PREPARÓ	SALIDA OBTENIDA
C5	Borrador con producto 2 (×2) y producto 4 (×1), al precio vigente.	VERDE `SIMULADO_APROBADO: todo coincide (simulación, no es una venta)`
C3	Producto 95 con `stock_visto = 3` ; se piden 10 unidades.	ROJO `RECHAZADO: producto 95: no alcanza el stock (pide 10, hay 3)`
C2a	Producto 5 a 111.923 centavos; el servidor sube el precio a 126.923 y se toma una instantánea nueva.	AMARILLO `CONFLICTO: producto 5: precio cambió de 111923 a 126923 centavos`
C2b	El cliente acepta: se actualiza el precio visto y se encola una solicitud **nueva** (UUID `2b…`) sobre el mismo borrador.	VERDE `SIMULADO_APROBADO` . La solicitud `2a…` queda en `CONFLICTO` como registro histórico.
C4	Producto 6 pasa a `activo = false` ; nueva instantánea y semáforo.	ROJO `RECHAZADO: producto 6: dado de baja` . En cache queda `disponible = 0` , `eliminado = 1` y sale del catálogo.
C6	Reenvío de la solicitud `50000000-…-0005` , que ya existe.	ERROR `UNIQUE constraint failed: cola_sync.solicitud_uuid` . Repetir el semáforo deja los cinco estados idénticos.

C3: el rojo es de cantidad

El producto existe y está disponible. La tablet no puede saber si esas 3 unidades siguen ahí: por eso no descuenta, sólo advierte.

C2: decide una persona

No hay nada roto: cambió el precio. El sistema informa; alguien frente al cliente decide si acepta pagar más. No se resuelve solo.

C6: la PK es local

Protege la tabla de la tablet. Si el dispositivo se reinicia y genera otro UUID, la protección desaparece: la defensa real vive en el servidor.

6.5 · La prueba de que el servidor no se movió

Antes de empezar se tomó una línea de base con conteos y dos huellas `md5` : una del vector `id:stock` y otra del vector `id:precio` de los 50.000 productos. Al terminar se comparó contra esa foto.

INVARIANTE	ANTES	DESPUÉS	VEREDICTO
`pedido` (filas)	200.000	200.000	IGUAL
`detalle_pedido` (filas)	200.000	200.000	IGUAL
`stock_total`	4.999.032	4.999.032	IGUAL
`stock_huella`	`95b86fa0…`	`95b86fa0…`	IGUAL
`precio_huella`	`1816a7c5…`	`c47b59be…`	CAMBIÓ

POR QUÉ LA ÚLTIMA FILA ES LA PRUEBA

El precio **tenía que cambiar**: es el caso C2. El stock **no podía cambiar**: el semáforo no descuenta. Que una huella se mueva y la otra no demuestra que la tablet **leyó y no escribió**.

Se usa `md5` del vector completo y no sólo la suma porque dos cambios opuestos —uno sube 3, otro baja 3— dejan la suma idéntica. La huella detecta cualquier cambio en cualquier fila, aunque el total no se mueva.

6.6 · Code review: cuatro hallazgos, cero cambios

#	HALLAZGO	DECISIÓN	POR QUÉ	CÓMO SE COMPROBÓ
H1	En U3 se descartó `pedido(forma_pago)` diciendo que el planner haría `Seq Scan` igual.	DESCARTADO con fundamento corregido	El planner **sí** usa el índice (`Bitmap Index Scan`), pero lee las mismas 1.471 páginas: el 33 % de las filas está en casi todas. No ahorra I/O y agrega costo de escritura.	Cardinalidad 33,53 / 33,29 / 33,18 %. Seq Scan 9,748 ms; con índice, `Heap Blocks: exact=1471` . Se creó en transacción y se revirtió: 0 índices quedaron.
H2	`REVOKE SELECT (precio)` no recorta un `GRANT SELECT` de tabla.	CORREGIDO	Un `REVOKE` por columna no puede quitar lo que un `GRANT` de tabla otorgó. Hay que revocar el privilegio de tabla y volver a otorgar por columnas.	`has_column_privilege` siguió en `true` tras el REVOKE. Con `REVOKE ALL` + `GRANT SELECT (columnas)` dio `false` .
H3	Un `GRANT` dentro de un bloque `DO` se ejecuta con los privilegios del invocador.	CORREGIDO	Un `DO` no cambia de usuario: si lo corre un superusuario, el `GRANT` se aplica de verdad. Una prueba de seguridad que usa `DO` no prueba nada.	Se crearon dos roles, el `DO` otorgó el rol y `pg_has_role` dio `true` . Se revocó y se eliminaron: 0 roles quedaron.
H4	No existe ninguna regla de stock suficiente en la base.	DEUDA documentada	Se pueden vender 999 unidades de un producto con stock 3. El semáforo lo detecta en la tablet, pero **la fuente de verdad no lo impide**.	`INSERT` de 999 unidades en transacción: entró sin error. `ROLLBACK` ; el stock quedó intacto.

POR QUÉ H4 NO SE CORRIGIÓ EN ESTE TP

Un trigger que compare `cantidad` contra `stock` sin bloquear la fila es una condición de carrera en espera: dos cajas leen el mismo stock 1 y lo venden las dos. La corrección real necesita `SELECT … FOR UPDATE` dentro de la transacción de venta y cambiar el contrato del servidor. Aplicarlo de apuro sería peor que documentarlo.

CIERRE DE LA REVISIÓN

Nada quedó modificado: **0 índices** `tp8` , **0 roles** `tp8` y **200.000 pedidos**. Los cuatro hallazgos se reproducen con un comando cada uno. Ese es el punto: **lo que no se ejecutó no se sabe**.

6.7 · Trazabilidad Kiro: cada requisito tiene su caso

REQUISITO (EARS)	DÓNDE SE IMPLEMENTA	PRUEBA
REQ-01 · conflicto de precio	`semaforo.sql` , motivo amarillo	C2: 111.923 → 126.923 centavos
REQ-02 · stock insuficiente	`semaforo.sql` , rojo #4	C3: pide 10, hay 3
REQ-03 · baja lógica	`disponible` / `eliminado` + FK `RESTRICT`	C4: producto 6, la fila queda
REQ-04 · idempotencia del reenvío	PK de `cola_sync` + `WHERE PENDIENTE`	C6: `UNIQUE constraint failed`

Evidencias U8: `movil/schema_sqlite.sql` · `movil/pruebas_sqlite.sql` · `db/tp8_v_snapshot_movil.sql` · `movil/cargar_snapshot.sql` · `movil/semaforo.sql` · `movil/casos/` · `movil/evidencias_casos.txt` · `db/tp8_verificacion_final.sql` · `db/tp8_code_review_verificaciones.sql` · `docs/kiro/` · `docs/informe.md` .

CAPÍTULO 7 · USO DE INTELIGENCIA ARTIFICIAL

07 La IA propone, la ejecución dispone

Kiro, OpenCode, cinco defectos que sólo aparecieron al correr y seis propuestas rechazadas.

El informe no declara «se usó IA» como un trámite: reconstruye el flujo completo. Cada pieza tiene una spec, un prompt, una salida generada, una verificación ejecutada y una decisión humana. La conclusión es incómoda y por eso vale: **una respuesta que parece razonable puede ser falsa de una forma que sólo la ejecución revela**.

7.1 · Un flujo, dos herramientas y una regla

1	2	3	4
ESPECIFICAR	GENERAR	EJECUTAR	DECIDIR
Kiro Requirements en EARS, diseño con alternativas descartadas y tasks en orden de construcción.	**OpenCode** Agente en terminal, modo Plan para revisar y modo ejecución para escribir scripts.	**Base real** Copia o transacción reversible, respaldo previo y salida guardada como evidencia.	**Humano** Se acepta, se corrige o se rechaza. La aprobación final no se delega.

HERRAMIENTA	PARA QUÉ SE USÓ	QUÉ NO SE LE DELEGÓ
Kiro	Specs de U8 y U5: requisitos, diseño, precedencia, idempotencia y tasks.	La decisión de arquitectura y el alcance.
OpenCode	Generación de SQL, PowerShell y Python; revisión en modo Plan.	La ejecución contra la base y la validación de resultados.

No se usó IA sobre datos personales. Los usuarios de la tablet son sintéticos (`caja-01`) y los clientes del catálogo son `Cliente N`.

LA REGLA QUE ORDENÓ TODO EL TRABAJO

Ningún script se ejecutó sin leerlo línea por línea, comprenderlo y probarlo en una copia o dentro de una transacción reversible. Cuando hubo cambio estructural, hubo `pg_dump` previo. **Entregar algo que no se puede explicar en la defensa equivale a no haberlo hecho.**

7.2 · Los defectos que sólo aparecieron al ejecutar

#	DEFECTO	CÓMO SE DETECTÓ	CORRECCIÓN APLICADA
1	`'t' = 1` **es falso en SQLite.** Los motivos «dado de baja» y «no disponible» nunca se disparaban; un producto discontinuado podía salir aprobado.	Corriendo C4: el estado esperado era `RECHAZADO` y el obtenido no. El error no lanzaba excepción.	Normalizar `'t'/'f' → 1/0` una sola vez al aterrizar la instantánea, más un control de dominio. Una frontera, no N comparaciones frágiles.
2	`.import --skip 1` **sobre una tabla inexistente** crea la tabla con la primera fila de datos como encabezado y pierde un producto.	El control de entrada dio `49.999` filas cuando el CSV tenía 50.001 líneas.	Crear `snapshot_servidor` *antes* del `.import`, con `schema_snapshot.sql`.
3	`INSERT … SELECT … ON CONFLICT` **es ambiguo** en SQLite: el parser no sabe si el `ON` pertenece al `INSERT` o al `SELECT`.	Error `near "DO": syntax error` al correr el UPSERT.	Agregar `WHERE true` antes del `ON CONFLICT`. No es decorativo: sin esa línea el script no corre.
4	**La justificación del índice de U3 era falsa.** Se había descartado diciendo que el planner haría `Seq Scan` igual.	`EXPLAIN` mostró `Bitmap Index Scan` con el índice creado.	Se mantuvo la decisión y se corrigió el argumento: lee las mismas 1.471 páginas y sólo agrega costo de escritura.
5	**No existe regla de stock en PostgreSQL.** Se pueden vender 999 unidades de un producto con stock 3.	`INSERT` de prueba dentro de una transacción: entró sin error.	Se documentó como deuda (H4). La corrección exige `SELECT … FOR UPDATE` y un cambio de contrato; no se improvisó.

LA HERRAMIENTA TAMBIÉN SE EQUIVOCA

El driver `simular_casos.ps1` tenía dos defectos propios: `Select-Object -First` cerraba el pipeline y cortaba la simulación después de C2, y la función `SQ` tomaba sólo el primer argumento, dejando secciones de evidencia vacías. Ninguno era un error de SQL: eran errores del programa que corría el SQL. **Una evidencia incompleta puede parecer una evidencia correcta**, por eso la corrida final se hizo desde cero y se verificaron las 348 líneas y los cinco estados.

7.3 · Lo que se propuso y se rechazó

PROPUESTA DE LA IA	POR QUÉ SE RECHAZÓ
`UNIQUE` en `cola_sync.id_local`	Impide modelar C2: se necesitan dos solicitudes sobre el mismo borrador, una en conflicto y otra aprobada.
Tabla `venta` en SQLite	Convertiría un `SIMULADO_APROBADO` en una venta confirmada. La tablet no puede confirmar nada.
Un solo booleano `activo` en la cache	Colapsa «dado de baja» y «no disponible»: dos incidentes distintos que el vendedor necesita distinguir.
`REAL` para los precios	Error de redondeo en la conciliación de caja. El dinero viaja en centavos enteros.
Resolver el conflicto de precio automáticamente	Alguien tiene que decidir si el cliente acepta pagar más. El sistema informa; no negocia.
Confiar en la PK de la tablet como idempotencia suficiente	Falla si el UUID cambia tras un reinicio. La protección de negocio va en el servidor, dentro de la transacción.

7.4 · Protocolo de datos y DUIA

- **Nada de datos personales.** Se trabajó con usuarios sintéticos y clientes seudónimos; nunca se enviaron nombres, mails, teléfonos ni la clave de la base.
- **Anonimización verificable.** En U5 se construyó `usuario_anon` con seudónimos deterministas y meses en lugar de fechas. La agregación por rol y mes devolvió **7 grupos idénticos** y un `EXCEPT` cruzado de **0 filas**.
- **Todo uso declarado.** La DUIA registra herramienta, finalidad, prompt o spec, qué se aceptó, qué se modificó y cómo se verificó.
- **Bitácora con errores reales.** No sólo aciertos: los cinco defectos de arriba y los dos del driver están documentados con su detección y corrección.

QUÉ CONTIENE LA DUIA

Parte A: esquema SQLite y pruebas.
Parte B: instantánea, carga y semáforo.
Parte C: casos C2–C6.
Parte D: code review.
Parte E: spec de Kiro y documentación.

Cada pieza tiene una spec, un archivo de evidencia y un commit propio. La trazabilidad se completa con `git log --oneline`: un commit por pieza, del esquema SQLite al cierre documental.

Evidencias IA: `docs/DUIA.md` · `docs/bitacora_ia.md` · `BITACORA.md` · `docs/evidencias_code_review.txt` · `movil/evidencias_casos.txt`.

CAPÍTULO 8 · CIERRE DE LA ENTREGA

08 Checklist, brechas y estado real

Diecinueve puntos verificables, tres observaciones abiertas y una brecha cerrada sobre la fecha de entrega.

La consigna es explícita: la entrega se aprueba sólo si demuestra **la totalidad** de los objetivos. Este capítulo los recorre uno por uno, indica dónde está la evidencia y marca sin maquillaje lo que todavía falta. Un informe que sólo dice «cumplido» no sirve para la defensa.

8.1 · Los catorce objetivos de las unidades 4 a 8

#	OBJETIVO	DÓNDE SE ACREDITA	ESTADO
1	Refactorización del esquema heredado validada contra las DF reales y llevada a BCNF.	§2.1–2.4 · `tp_fnbc_control_lote.sql`	CUMPLE
2	Migración sin pérdida en transacciones reversibles, especificada en Kiro y con plan de reversión.	§2.4 · vista de compatibilidad + doble `EXCEPT`	PARCIAL
3	Desnormalización controlada documentada: cuándo, por qué y con qué salvaguardas.	§2.5 · `tp_desnormalizacion_top_categorias.sql`	CUMPLE
4	Esquema de usuarios, roles y permisos con mínimo privilegio.	§3.1–3.2 · `db/roles.sql` · `verificacion_roles.txt`	CUMPLE
5	Auditoría de accesos y operaciones probada con simulacro reconstruido desde el log y contención.	§3.3–3.4 · `auditoria_test.sql` · `informe_incidente.md`	CUMPLE
6	Auditoría de seguridad del esquema asistida por IA con decisión humana documentada.	§3.5 · tabla de propuestas aceptadas y rechazadas	CUMPLE
7	Plan de mantenimiento completo, automatizado con OpenCode y documentado.	§4.1–4.2 · `backup_foodstore.ps1` · `restore_test.ps1`	CUMPLE
8	Simulacro de desastre contra reloj con informe post-incidente.	§4.3–4.4 · RTO 48,3 s · RPO ~1,5 h · `informe_u6.pdf`	CUMPLE
9	Modelado documental del dominio con consultas verificadas sobre datos reales.	§5.1–5.3 · `consultas_mongodb.js` · `salida_mongodb.txt`	CUMPLE
10	Comparación documental vs. relacional y justificación de la tecnología elegida.	§5.4 · `consultas_postgresql.sql` · `informe_u7_nosql.md`	CUMPLE
11	Base SQLite con esquema y sincronización especificados en Kiro y generados con OpenCode.	§6.1–6.3 · `movil/` · `docs/kiro/`	CUMPLE
12	Sincronización simulada con casos de conflicto probados y resolución demostrada.	§6.4 · C2a/C2b/C3/C4/C5/C6 · `evidencias_casos.txt`	CUMPLE
13	Code review final asistido por IA con decisión humana sobre cada hallazgo.	§6.6 · H1–H4 · `evidencias_code_review.txt`	CUMPLE
14	Capa de datos completa con specs, scripts versionados en Git, mantenimiento y bitácora.	§1–§7 · commits del TP 8 en `main`, uno por pieza	CUMPLE

8.2 · Los cinco contenidos exigidos al informe técnico

#	QUÉ PIDE LA CONSIGNA	DÓNDE SE RESPONDE
15	Qué se implementó en cada unidad (4 a 8).	Capítulos 2 a 6, con evidencia por sección.
16	Cómo se probó: transacciones reversibles, copias, simulacros y casos de conflicto.	§2.4, §3.4, §4.2–4.3, §6.4–6.5.
17	Resultados, incluido el tiempo de restauración y el post-incidente.	§4.3: RTO 48,3 s; RPO ~1,5 h; informe post-incidente en §4.4.
18	Decisiones de diseño: refactorización, desnormalización, roles y elección documental/relacional.	§2.4–2.5, §3.1, §5.4, §6.2.
19	Herramientas de IA, prompts o specs, propuestas aceptadas y descartadas.	Capítulo 7 y `docs/DUIA.md` + `bitacora_ia.md`.

8.3 · Cuatro observaciones, una ya corregida

O1 · El TP 8 estaba sin commitear RESUELTO

Los scripts de U8, `movil/`, `docs/kiro/`, la DUIA y la bitácora figuraban como no versionados, y la DUIA enumeraba commits que no existían en el historial. Se separó el trabajo en **un commit por pieza** —esquema, vista, carga, semáforo, casos, invariantes, review, specs, bitácora, entrega y DUIA— y la DUIA se actualizó a lo que dice `git log`.

VERIFICADO Árbol limpio y `main` igual a `origin/main`.

O2 · U4 sin spec formal de Kiro

La migración y el plan de reversión están documentados en el informe y en `normalizacion.md`, pero no existe un `specs/` de U4 equivalente al de U8.

PARCIAL La consigna pide «especificada en Kiro».

O3 · Discrepancia de conteo en U6

El informe de U6 declara `detalle_pedido` con 450.000 filas y 15 categorías; la base documentada y el resto de las mediciones hablan de **200.000** y **10**. Es una inconsistencia de documentación —el simulacro parece haber corrido sobre otra copia—, no de datos.

CORREGIR Unificar las cifras antes de la defensa.

O4 · El TP 8 del segundo repositorio es una variante anterior

Su DUIA lista nombres que no coinciden con el equipo y su esquema no incluye la tabla de aterrizaje. Se toma como complemento, no como fuente canónica.

ACLARAR El informe se apoya en el repositorio principal.

8.4 · Qué queda cerrado y qué queda abierto

CIERRE DE LA CAPA DE DATOS

Cerrado: el esquema relacional en BCNF con una desnormalización auditada; los roles con mínimo privilegio y su verificación real; el respaldo restaurado en 48,3 segundos; el documento de pedido con consultas equivalentes; la base de la tablet con un semáforo que distingue cuatro rojos y un amarillo; y el TP 8 versionado con un commit por pieza y su DUIA alineada al historial. Y una prueba que vale más que cualquier captura: **PostgreSQL quedó idéntico, con huella md5**.

Abierto: la regla de stock en el servidor (H4) y la protección de idempotencia del lado del servidor. Ninguno se oculta: se documentan como deuda con su motivo y su camino de corrección.

> *«La tablet no puede confirmar nada. Su mejor resultado posible se llama SIMULADO_APROBADO, y no es una venta».*

Documento base: `Condiciones de entrega · TPI segunda entrega` · **Repositorios:** `02_Base-de-datos-II` y `Proyecto_Food_Store_MongoDB_SqlLite`.

ANEXO · REPRODUCIBILIDAD

A Dónde está cada cosa y cómo se reproduce

Repositorios, inventario de evidencia y entorno de ejecución.

A.1 · Repositorios del equipo

REPOSITORIO	QUÉ CONTIENE	ESTADO
`ismaSaleme12/02_Base-de-datos-II`	PostgreSQL: esquema, U4, U5, U6 y TP 8 completo (SQLite, semáforo, casos, Kiro, DUIA).	CANÓNICO
`ismaSaleme12/Proyecto_Food_Store_MongoDB_SqlLite`	U7 documental y una variante anterior de U8 (`TP-8/`) con simulador en Python.	COMPLEMENTO

A.2 · Inventario de evidencia por unidad

UNIDAD	ARCHIVOS PRINCIPALES	QUÉ DEMUESTRA
U4	`db/tp_fnbc_control_lote.sql` · `db/tp_desnormalizacion_top_categorias.sql` · `docs/normalizacion.md` · `Informe_TP_Unidad4…pdf`	BCNF, migración sin pérdida, backfill de 3.660 filas y reporte 73,9 ms → 0,23 ms.
U5	`db/roles.sql` · `db/auditoria_test.sql` · `db/simulacro_incidente.sql` · `db/usuario_anon.sql` · `docs/verificacion_roles.txt`	Seis roles, permisos por columna, log probado, incidente reconstruido y anonimización con 0 diferencias.
U6	`SPEC_U6.md` · `backups/*.ps1` · `db/health_check.sql` · `db/maintenance.sql` · `docs/informe_u6.pdf`	Respaldo, salud, mantenimiento y restauración medida: 48,3 s, dump de 18,4 MB.
U7	`consultas_mongodb.js` · `consultas_postgresql.sql` · `datos_migracion.json` · `salida_mongodb.txt`	10 documentos, cuatro consultas, validadores e índices y comparación de IDs con PostgreSQL.
U8	`movil/schema_sqlite.sql` · `movil/semaforo.sql` · `movil/casos/` · `db/tp8_*.sql` · `docs/kiro/`	Cuatro tablas `STRICT` , seis casos, huellas invariantes y cuatro hallazgos de review.

A.3 · Entorno y reproducción mínima

POSTGRESQL	17.10 · base `Food_Store_Copia`
SQLITE	3.50.4 · tablas `STRICT`
MONGODB	instancia local · base documental de pedidos
DATOS	10 categorías · 20.000 clientes · 50.000 productos · 200.000 pedidos

El TP 8 se reproduce en orden: respaldo → línea de base → vista → exportación CSV → esquema SQLite → aterrizaje → carga normalizada → semáforo → casos → verificación final. Cada paso depende del anterior y está detallado en el `README.md` del repositorio.

```
# 1. Verificar que la base esté cargada
psql -d Food_Store_Copia -tAc "select count(*) from producto;"

# 2. Tomar la línea de base de los invariantes
psql -d Food_Store_Copia -f db/tp8_linea_base.sql

# 3. Crear la instantánea y exportarla
psql -d Food_Store_Copia -f db/tp8_v_snapshot_movil.sql
# \copy (SELECT * FROM v_snapshot_movil) TO ... CSV HEADER

# 4. Correr los casos y verificar que nada se movió
.\movil\simular_casos.ps1
psql -d Food_Store_Copia -f db/tp8_verificacion_final.sql
```

CRITERIO DE EVIDENCIA

No se incluyen capturas sueltas. Cada afirmación del informe apunta a un script, una salida real o una medición que puede volver a ejecutarse. Un resultado sin comando que lo reproduzca no se considera evidencia.

A.4 · Glosario mínimo para la defensa

TÉRMINO	QUÉ SIGNIFICA	DÓNDE SE USA
BCNF	Todo determinante de una dependencia funcional es superclave.	U4 · `control_lote_almacen`
RTO	Tiempo máximo aceptable para restaurar el servicio tras un desastre.	U6 · observado 48,3 s
RPO	Cantidad máxima de datos que se acepta perder, medida como antigüedad del respaldo.	U6 · observado ~1,5 h
EARS	Notación de requisitos con disparadores *WHEN / IF / WHILE* y respuesta obligatoria.	U8 · `requirements.md`
DUIA	Declaración de uso de IA: herramienta, finalidad, prompts y verificación.	Capítulo 7 · `docs/DUIA.md`
STRICT	Modo de tabla SQLite que valida el tipo de cada columna en la escritura.	U8 · cuatro tablas
UPSERT	Insertar o actualizar si la clave ya existe (`ON CONFLICT DO UPDATE`).	U8 · carga de la cache
Idempotencia	Repetir la operación no cambia el resultado ni duplica el efecto.	U8 · semáforo y C6

Food Store · Base de Datos II · TPI segunda entrega · Sabrina Gimenez · Ismael Saleme · Jeronimo Coronel · Joaquin Godoy · Unidades 4 a 8 · 2026.